

Internet Engineering Task Force (IETF)
Request for Comments: 7919
Updates: 2246, 4346, 4492, 5246
Category: Standards Track
ISSN: 2070-1721

D. Gillmor
ACLU
August 2016

Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS)

Abstract

Traditional finite-field-based Diffie-Hellman (DH) key exchange during the Transport Layer Security (TLS) handshake suffers from a number of security, interoperability, and efficiency shortcomings. These shortcomings arise from lack of clarity about which DH group parameters TLS servers should offer and clients should accept. This document offers a solution to these shortcomings for compatible peers by using a section of the TLS "Supported Groups Registry" (renamed from "EC Named Curve Registry" by this document) to establish common finite field DH parameters with known structure and a mechanism for peers to negotiate support for these groups.

This document updates TLS versions 1.0 (RFC 2246), 1.1 (RFC 4346), and 1.2 (RFC 5246), as well as the TLS Elliptic Curve Cryptography (ECC) extensions (RFC 4492).

Status of This Memo

This is an Internet Standards Track document.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 7841.

Information about the current status of this document, any errata, and how to provide feedback on it may be obtained at <http://www.rfc-editor.org/info/rfc7919>.

Copyright Notice

Copyright (c) 2016 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<http://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Simplified BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Simplified BSD License.

Table of Contents

- 1. Introduction 4
 - 1.1. Requirements Language 5
 - 1.2. Vocabulary 5
- 2. Named Group Overview 5
- 3. Client Behavior 6
 - 3.1. Client Local Policy on Custom Groups 7
- 4. Server Behavior 8
- 5. Optimizations 9
 - 5.1. Checking the Peer's Public Key 9
 - 5.2. Short Exponents 9
 - 5.3. Table Acceleration 10
- 6. Operational Considerations 10
 - 6.1. Preference Ordering 10
- 7. IANA Considerations 11
- 8. Security Considerations 12
 - 8.1. Negotiation Resistance to Active Attacks 12
 - 8.2. Group Strength Considerations 13
 - 8.3. Finite Field DHE Only 13
 - 8.4. Deprecating Weak Groups 14
 - 8.5. Choice of Groups 14
 - 8.6. Timing Attacks 14
 - 8.7. Replay Attacks from Non-negotiated FFDHE 15
 - 8.8. Forward Secrecy 15
 - 8.9. False Start 15
- 9. Privacy Considerations 16
 - 9.1. Client Fingerprinting 16
- 10. References 16
 - 10.1. Normative References 16
 - 10.2. Informative References 17
- Appendix A. Supported Groups Registry (Formerly "EC Named Curve Registry") 19
 - A.1. ffdhe2048 19
 - A.2. ffdhe3072 20
 - A.3. ffdhe4096 22
 - A.4. ffdhe6144 23
 - A.5. ffdhe8192 26
- Acknowledgements 29
- Author's Address 29

1. Introduction

Traditional TLS [RFC5246] offers a Diffie-Hellman Ephemeral (DHE) key exchange mode that provides forward secrecy for the connection. The client offers a cipher suite in the ClientHello that includes DHE, and the server offers the client group parameters generator g and modulus p . If the client does not consider the group strong enough (e.g., p is too small, p is not prime, or there are small subgroups that cannot be easily avoided) or if it is unable to process the group for other reasons, the client has no recourse but to terminate the connection.

Conversely, when a TLS server receives a suggestion for a DHE cipher suite from a client, it has no way of knowing what kinds of DH groups the client is capable of handling or what the client's security requirements are for this key exchange session. For example, some widely distributed TLS clients are not capable of DH groups where $p > 1024$ bits. Other TLS clients may, by policy, wish to use DHE only if the server can offer a stronger group (and are willing to use a non-PFS (Perfect Forward Secrecy) key exchange mechanism otherwise). The server has no way of knowing which type of client is connecting but must select DH parameters with insufficient knowledge.

Additionally, the DH parameters selected by the server may have a known structure that renders them secure against a small subgroup attack, but a client receiving an arbitrary p and g has no efficient way to verify that the structure of a new group is reasonable for use.

This modification to TLS solves these problems by using a section of the "Supported Groups Registry" (renamed from "EC Named Curve Registry" by this document) to select common DH groups with known structure and defining the use of the "elliptic_curves(10)" extension (described here as the Supported Groups extension) for clients advertising support for DHE with these groups. This document also provides guidance for compatible peers to take advantage of the additional security, availability, and efficiency offered.

The use of this mechanism by one compatible peer when interacting with a non-compatible peer should have no detrimental effects.

This document updates TLS versions 1.0 [RFC2246], 1.1 [RFC4346], and 1.2 [RFC5246], as well as the TLS ECC extensions [RFC4492].

1.1. Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

1.2. Vocabulary

The terms "DHE" or "FFDHE" are used in this document to refer to the finite-field-based Diffie-Hellman ephemeral key exchange mechanism in TLS. TLS also supports Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) key exchanges [RFC4492], but this document does not document their use. A registry previously used only by ECDHE-capable implementations is expanded in this document to cover FFDHE groups as well. "FFDHE cipher suites" is used in this document to refer exclusively to cipher suites with FFDHE key exchange mechanisms, but note that these suites are typically labeled with a TLS_DHE_ prefix.

2. Named Group Overview

We use previously unallocated codepoints within the extension currently known as "elliptic_curves" (Section 5.1.1. of [RFC4492]) to indicate known finite field groups. The extension's semantics are expanded from "Supported Elliptic Curves" to "Supported Groups". The enum datatype used in the extension has been renamed from NamedCurve to NamedGroup. Its semantics are likewise expanded from "named curve" to "named group".

Additionally, we explicitly relax the requirement about when the Supported Groups extension can be sent. This extension MAY be sent by the client when either FFDHE or ECDHE cipher suites are listed.

Codepoints in the "Supported Groups Registry" with a high byte of 0x01 (that is, between 256 and 511, inclusive) are set aside for FFDHE groups, though only a small number of them are initially defined and we do not expect many other FFDHE groups to be added to this range. No codepoints outside of this range will be allocated to FFDHE groups. The new codepoints for the "Supported Groups Registry" are:

```
enum {  
  // other already defined elliptic curves (see RFC 4492)  
    ffdhe2048(256), ffdhe3072(257), ffdhe4096(258),  
    ffdhe6144(259), ffdhe8192(260),  
  //  
} NamedGroup;
```

These additions to the "Supported Groups Registry" are described in detail in Appendix A. They are all safe primes derived from the base of the natural logarithm ("e"), with the high and low 64 bits set to 1 for efficient Montgomery or Barrett reduction.

The use of the base of the natural logarithm here is as a "nothing-up-my-sleeve" number. The goal is to guarantee that the bits in the middle of the modulus are effectively random, while avoiding any suspicion that the primes have secretly been selected to be weak according to some secret criteria. [RFC3526] used pi for this value. See Section 8.5 for reasons that this document does not reuse pi.

3. Client Behavior

A TLS client that is capable of using strong finite field Diffie-Hellman groups can advertise its capabilities and its preferences for stronger key exchange by using this mechanism.

The compatible client that wants to be able to negotiate strong FFDHE sends a Supported Groups extension (identified by type `elliptic_curves(10)` in [RFC4492]) in the ClientHello and includes a list of known FFDHE groups in the extension data, ordered from most preferred to least preferred. If the client also supports and wants to offer ECDHE key exchange, it MUST use a single Supported Groups extension to include all supported groups (both ECDHE and FFDHE groups). The ordering SHOULD be based on client preference, but see Section 6.1 for more nuance.

A client that offers a Supported Groups extension containing an FFDHE group SHOULD also include at least one FFDHE cipher suite in the ClientHello.

A client that offers a group MUST be able and willing to perform a DH key exchange using that group.

A client that offers one or more FFDHE groups in the Supported Groups extension and an FFDHE cipher suite and that receives an FFDHE cipher suite from the server SHOULD take the following steps upon receiving the ServerKeyExchange:

- o For non-anonymous cipher suites where the offered certificate is valid and appropriate for the peer, validate the signature over the ServerDHParams. If not valid, terminate the connection.

- o If the signature over ServerDHParams is valid, compare the selected `dh_p` and `dh_g` with the FFDHE groups offered by the client. If none of the offered groups match, the server is not compatible with this document. The client MAY decide to continue the connection if the selected group is acceptable under local policy, or it MAY decide to terminate the connection with a fatal `insufficient_security(71)` alert.
- o If the client continues (either because the server offered a matching group or because local policy permits the offered custom group), the client MUST verify that `dh_Ys` is in the range $1 < dh_Ys < dh_p - 1$. If `dh_Ys` is not in this range, the client MUST terminate the connection with a fatal `handshake_failure(40)` alert.
- o If `dh_Ys` is in range, then the client SHOULD continue with the connection as usual.

3.1. Client Local Policy on Custom Groups

Compatible clients that are willing to accept FFDHE cipher suites from non-compatible servers may have local policy about what custom FFDHE groups they are willing to accept. This local policy presents a risk to clients, who may accept weakly protected communications from misconfigured servers.

This document cannot enumerate all possible safe local policy (the safest may be to simply reject all custom groups), but compatible clients that accept some custom groups from the server MUST do at least cursory checks on group size and may take other properties into consideration as well.

A compatible client that accepts FFDHE cipher suites using custom groups from non-compatible servers MUST reject any group with `|dh_p| < 768` bits and SHOULD reject any group with `|dh_p| < 1024` bits.

A compatible client that rejects a non-compatible server's custom group may decide to retry the connection while omitting all FFDHE cipher suites from the ClientHello. A client SHOULD only use this approach if it successfully verified the server's expected signature over the ServerDHParams, to avoid being forced by an active attacker into a non-preferred cipher suite.

4. Server Behavior

If a compatible TLS server receives a Supported Groups extension from a client that includes any FFDHE group (i.e., any codepoint between 256 and 511, inclusive, even if unknown to the server), and if none of the client-proposed FFDHE groups are known and acceptable to the server, then the server **MUST NOT** select an FFDHE cipher suite. In this case, the server **SHOULD** select an acceptable non-FFDHE cipher suite from the client's offered list. If the extension is present with FFDHE groups, none of the client's offered groups are acceptable by the server, and none of the client's proposed non-FFDHE cipher suites are acceptable to the server, the server **MUST** end the connection with a fatal TLS alert of type `insufficient_security`(71).

If at least one FFDHE cipher suite is present in the client cipher suite list and the Supported Groups extension is either absent from the ClientHello entirely or contains no FFDHE groups (i.e., no codepoints between 256 and 511, inclusive), then the server knows that the client is not compatible with this document. In this scenario, a server **MAY** select a non-FFDHE cipher suite, or it **MAY** select an FFDHE cipher suite and offer an FFDHE group of its choice to the client as part of a traditional ServerKeyExchange.

A compatible TLS server that receives the Supported Groups extension with FFDHE codepoints in it and that selects an FFDHE cipher suite **MUST** select one of the client's offered groups. The server indicates the choice of group to the client by sending the group's parameters as usual in the ServerKeyExchange as described in Section 7.4.3 of [RFC5246].

A TLS server **MUST NOT** select a named FFDHE group that was not offered by a compatible client.

A TLS server **MUST NOT** select an FFDHE cipher suite if the client did not offer one, even if the client offered an FFDHE group in the Supported Groups extension.

If a non-anonymous FFDHE cipher suite is selected and the TLS client has used this extension to offer an FFDHE group of comparable or greater strength than the server's public key, the server **SHOULD** select an FFDHE group at least as strong as the server's public key. For example, if the server has a 3072-bit RSA key and the client offers only `ffdhe2048` and `ffdhe4096`, the server **SHOULD** select `ffdhe4096`.

When an FFDHE cipher suite is selected and the client sends a ClientKeyExchange, the server MUST verify that $1 < dh_Yc < dh_p - 1$. If dh_Yc is out of range, the server MUST terminate the connection with a fatal handshake_failure(40) alert.

5. Optimizations

In a key exchange with a successfully negotiated known FFDHE group, both peers know that the group in question uses a safe prime as a modulus and that the group in use is of size $p-1$ or $(p-1)/2$. This allows at least three optimizations that can be used to improve performance.

5.1. Checking the Peer's Public Key

Peers MUST validate each other's public key Y (dh_Ys offered by the server or dh_Yc offered by the client) by ensuring that $1 < Y < p-1$. This simple check ensures that the remote peer is properly behaved and isn't forcing the local system into the 2-element subgroup.

To reach the same assurance with an unknown group, the client would need to verify the primality of the modulus, learn the factors of $p-1$, and test both the generator g and Y against each factor to avoid small subgroup attacks.

5.2. Short Exponents

Traditional finite field Diffie-Hellman has each peer choose their secret exponent from the range $[2, p-2]$. Using exponentiation by squaring, this means each peer must do roughly $2 \cdot \log_2(p)$ multiplications, twice (once for the generator and once for the peer's public key).

Peers concerned with performance may also prefer to choose their secret exponent from a smaller range, doing fewer multiplications, while retaining the same level of overall security. Each named group indicates its approximate security level and provides a lower bound on the range of secret exponents that should preserve it. For example, rather than doing $2 \cdot 2 \cdot 3072$ multiplications for an ffdhe3072 handshake, each peer can choose to do $2 \cdot 2 \cdot 275$ multiplications by choosing their secret exponent from the range $[2^{274}, 2^{275}]$ (that is, an m -bit integer where m is at least 275) and still keep the same approximate security level.

A similar short-exponent approach is suggested in a Secure SHell (SSH) Diffie-Hellman key exchange (see Section 6.2 of [RFC4419]).

5.3. Table Acceleration

Peers wishing to further accelerate FFDHE key exchange can also pre-compute a table of powers of the generator of a known group. This is a memory vs. time trade-off, and it only accelerates the first exponentiation of the ephemeral DH exchange (the fixed-base exponentiation). The variable-base exponentiation (using the peer's public exponent as a base) still needs to be calculated as normal.

6. Operational Considerations

6.1. Preference Ordering

The ordering of named groups in the Supported Groups extension may contain some ECDHE groups and some FFDHE groups. These SHOULD be ranked in the order preferred by the client.

However, the ClientHello also contains a list of desired cipher suites, also ranked in preference order. This presents the possibility of conflicted preferences. For example, if the ClientHello contains a cipher_suite field with two choices in order <TLS_DHE_RSA_WITH_AES_128_CBC_SHA, TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA> and the Supported Groups extension contains two choices in order <secp256r1,ffdhe3072>, then there is a clear contradiction. Clients SHOULD NOT present such a contradiction since it does not represent a sensible ordering. A server that encounters such a contradiction when selecting between an ECDHE or FFDHE key exchange mechanism while trying to respect client preferences SHOULD give priority to the Supported Groups extension (in the example case, it should select TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA with secp256r1) but MAY resolve the contradiction any way it sees fit.

More subtly, clients MAY interleave preferences between ECDHE and FFDHE groups; for example, if stronger groups are preferred regardless of cost, but weaker groups are acceptable, the Supported Groups extension could consist of <ffdhe8192,secp384p1,ffdhe3072,secp256r1>. In this example, with the same cipher_suite field offered as the previous example, a server configured to respect client preferences and with support for all listed groups SHOULD select TLS_DHE_RSA_WITH_AES_128_CBC_SHA with ffdhe8192. A server configured to respect client preferences and with support for only secp384p1 and ffdhe3072 SHOULD select TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA with secp384p1.

7. IANA Considerations

This document renames the "EC Named Curve Registry" (originally defined in [RFC4492] and updated by [RFC7027]) to the "Supported Groups Registry". See <<https://www.iana.org/assignments/tls-parameters>>.

This document expands the semantics of this registry to include groups based on finite fields in addition to groups based on elliptic curves. IANA has added a range designation to the registry, indicating that values from 256-511 (inclusive) are set aside for "Finite Field Diffie-Hellman groups" and that all other entries in the registry are "Elliptic curve groups".

This document allocates five well-defined codepoints in the registry for specific finite field Diffie-Hellman groups defined in Appendix A.

In addition, the four highest codepoints in the range 508-511, inclusive, are designated for Private Use [RFC5226] by peers who have privately developed finite field Diffie-Hellman groups that they wish to signal internally.

The updated registry section is as follows:

Value	Description	DTLS-OK	Reference
256	ffdhe2048	Y	RFC 7919
257	ffdhe3072	Y	RFC 7919
258	ffdhe4096	Y	RFC 7919
259	ffdhe6144	Y	RFC 7919
260	ffdhe8192	Y	RFC 7919
508-511	Private Use	-	RFC 7919

IANA has also renamed the "elliptic_curves" extension as "supported_groups" and added a reference to this document in the "ExtensionType Values" registry. See <<http://www.iana.org/assignments/tls-extensiontype-values>>.

8. Security Considerations

8.1. Negotiation Resistance to Active Attacks

Because the contents of the Supported Groups extension are hashed in the Finished message, an active Man in the Middle (MITM) that tries to filter or omit groups will cause the handshake to fail, but possibly not before getting the peer to do something it would not otherwise have done.

An attacker who impersonates the server can try to do any of the following:

- o Pretend that a non-compatible server is actually capable of this extension and select a group from the client's list, causing the client to select a group it is willing to negotiate. It is unclear how this would be an effective attack.
- o Pretend that a compatible server is actually non-compatible by negotiating a non-FFDHE cipher suite. This is no different than MITM cipher suite filtering.
- o Pretend that a compatible server is actually non-compatible by negotiating a DHE cipher suite with a custom (perhaps weak) group selected by the attacker. This is no worse than the current scenario and would require the attacker to be able to sign the ServerDHParams, which should not be possible without access to the server's secret key.

An attacker who impersonates the client can try to do the following:

- o Pretend that a compatible client is not compatible (e.g., by not offering the Supported Groups extension or by replacing the Supported Groups extension with one that includes no FFDHE groups). This could cause the server to negotiate a weaker DHE group during the handshake or to select a non-FFDHE cipher suite, but it would fail to complete during the final check of the Finished message.
- o Pretend that a non-compatible client is compatible (e.g., by adding the Supported Groups extension or by adding FFDHE groups to the extension). This could cause the server to select a particular named group in the ServerKeyExchange or to avoid selecting an FFDHE cipher suite. The peers would fail to compute the final check of the Finished message.

- o Change the list of groups offered by the client (e.g., by removing the stronger of the set of groups offered). This could cause the server to negotiate a weaker group than desired, but again, should be caught by the check in the Finished message.

8.2. Group Strength Considerations

TLS implementations using FFDHE key exchange should consider the strength of the group they negotiate. The strength of the selected group is one of the factors that define the connection's resilience against attacks on the session's confidentiality and integrity, since the session keys are derived from the DHE handshake.

While attacks on integrity must generally happen while the session is in progress, attacks against session confidentiality can happen significantly later if the entire TLS session is stored for offline analysis. Therefore, FFDHE groups should be selected by clients and servers based on confidentiality guarantees they need. Sessions that need extremely long-term confidentiality should prefer stronger groups.

[ENISA] provides rough estimates of group resistance to attack and recommends that forward-looking implementations ("future systems") should use FFDHE group sizes of at least 3072 bits. `ffdhe3072` is intended for use in these implementations.

Other sources (e.g., [NIST]) estimate the security levels of the Discrete Log (DLOG) problem to be slightly more difficult than [ENISA]. This document's suggested minimum exponent sizes in Appendix A for implementations that use the short-exponent optimization (Section 5.2) are deliberately conservative to account for the range of these estimates.

8.3. Finite Field DHE Only

Note that this document specifically targets only finite field Diffie-Hellman ephemeral key exchange mechanisms. It does not cover the non-ephemeral DH key exchange mechanisms, nor does it address ECDHE key exchange, which is defined in [RFC4492].

Measured by computational cost to the TLS peers, ECDHE appears today to offer a much stronger key exchange mechanism than FFDHE.

8.4. Deprecating Weak Groups

Advances in hardware or in finite field cryptanalysis may cause some of the negotiated groups to not provide the desired security margins, as indicated by the estimated work factor of an adversary to discover the premaster secret (and may therefore compromise the confidentiality and integrity of the TLS session).

Revisions of this document should mark known weak groups as explicitly deprecated for use in TLS and should update the estimated work factor needed to break the group if the cryptanalysis has changed. Implementations that require strong confidentiality and integrity guarantees should avoid using deprecated groups and should be updated when the estimated security margins are updated.

8.5. Choice of Groups

Other lists of named finite field Diffie-Hellman groups [STRONGSWAN-IKE] exist. This document chooses to not reuse them for several reasons:

- o Using the same groups in multiple protocols increases the value for an attacker with the resources to crack any single group.
- o The Internet Key Exchange Protocol (IKE) groups include weak groups like MODP768 that are unacceptable for secure TLS traffic.
- o Mixing group parameters across multiple implementations leaves open the possibility of some sort of cross-protocol attack. This shouldn't be relevant for ephemeral scenarios, and even with non-ephemeral keying, services shouldn't share keys; however, using different groups avoids these failure modes entirely.

8.6. Timing Attacks

Any implementation of finite field Diffie-Hellman key exchange should use constant-time modular-exponentiation implementations. This is particularly true for those implementations that ever reuse DHE secret keys (so-called "semi-static" ephemeral keying) or share DHE secret keys across a multiple machines (e.g., in a load-balancer situation).

8.7. Replay Attacks from Non-negotiated FFDHE

[SECURE-RESUMPTION], [CROSS-PROTOCOL], and [SSL3-ANALYSIS] all show a malicious peer using a bad FFDHE group to maneuver a client into selecting a premaster secret of the peer's choice, which can be replayed to another server using a non-FFDHE key exchange and can then be bootstrapped to replay client authentication.

To prevent this attack (barring the session hash fix documented in [RFC7627]), a client would need not only to implement this document, but also to reject non-negotiated FFDHE cipher suites whose group structure it cannot afford to verify. Such a client would need to abort the initial handshake and reconnect to the server in question without listing any FFDHE cipher suites on the subsequent connection.

This trade-off may be too costly for most TLS clients today but may be a reasonable choice for clients performing client certificate authentication or for clients who have other reasons to be concerned about server-controlled premaster secrets.

8.8. Forward Secrecy

One of the main reasons to prefer FFDHE ciphersuites is forward secrecy, the ability to resist decryption even if the endpoint's long-term secret key (usually RSA) is revealed in the future.

This property depends on both sides of the connection discarding their ephemeral keys promptly. Implementations should wipe their FFDHE secret key material from memory as soon as it is no longer needed and should never store it in persistent storage.

Forward secrecy also depends on the strength of the Diffie-Hellman group; using a very strong symmetric cipher like AES256 with a forward-secret cipher suite but generating the keys with a much weaker group like `dhe2048` simply moves the adversary's cost from attacking the symmetric cipher to attacking the `dh_Ys` or `dh_Yc` ephemeral key shares.

If the goal is to provide forward secrecy, attention should be paid to all parts of the cipher suite selection process, both key exchange and symmetric cipher choice.

8.9. False Start

Clients capable of TLS False Start [RFC7918] may receive a proposed FFDHE group from a server that is attacker controlled. In particular, the attacker can modify the ClientHello to strip the proposed FFDHE groups, which may cause the server to offer a weaker

FFDHE group than it should, and this will not be detected until receipt of the server's Finished message. This could cause a client using the False Start protocol modification to send data encrypted under a weak key agreement.

Clients should have their own classification of FFDHE groups that are "cryptographically strong" in the same sense described in the description of symmetric ciphers in [RFC7918] and SHOULD offer at least one of these in the initial handshake if they contemplate using the False Start protocol modification with an FFDHE cipher suite.

Compatible clients performing a full handshake MUST NOT use the False Start protocol modification if the server selects an FFDHE cipher suite but sends a group that is not cryptographically strong from the client's perspective.

9. Privacy Considerations

9.1. Client Fingerprinting

This extension provides a few additional bits of information to distinguish between classes of TLS clients (e.g., see [PANOPTICCLICK]). To minimize this sort of fingerprinting, clients SHOULD support all named groups at or above their minimum security threshold. New groups SHOULD NOT be added to the "Supported Groups Registry" without consideration of the cost of browser fingerprinting.

10. References

10.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<http://www.rfc-editor.org/info/rfc2119>>.
- [RFC4492] Blake-Wilson, S., Bolyard, N., Gupta, V., Hawk, C., and B. Moeller, "Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS)", RFC 4492, DOI 10.17487/RFC4492, May 2006, <<http://www.rfc-editor.org/info/rfc4492>>.
- [RFC5226] Narten, T. and H. Alvestrand, "Guidelines for Writing an IANA Considerations Section in RFCs", BCP 26, RFC 5226, DOI 10.17487/RFC5226, May 2008, <<http://www.rfc-editor.org/info/rfc5226>>.

- [RFC5246] Dierks, T. and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2", RFC 5246, DOI 10.17487/RFC5246, August 2008, <<http://www.rfc-editor.org/info/rfc5246>>.
- [RFC7918] Langley, A., Modadugu, N., and B. Moeller, "Transport Layer Security (TLS) False Start", DOI 10.17487/RFC7918, June 2016, <<http://www.rfc-editor.org/info/rfc7918>>.

10.2. Informative References

- [CROSS-PROTOCOL] Mavrogiannopolous, N., Vercauteren, F., Velichkov, V., and B. Preneel, "A Cross-Protocol Attack on the TLS Protocol", In Proceedings of the 2012 ACM Conference on Computer and Communications Security, DOI 10.1145/2382196.2382206, October 2012, <<http://www.cosic.esat.kuleuven.be/publications/article-2216.pdf>>.
- [ECRYPTII] European Network of Excellence in Cryptology II, "ECRYPT II Yearly Report on Algorithms and Keysizes (2011-2012)", Revision 1.0, September 2012, <<http://www.ecrypt.eu.org/ecrypt2/documents/D.SPA.20.pdf>>.
- [ENISA] European Union Agency for Network and Information Security (ENISA), "Algorithms, Key Sizes and Parameters Report - 2014", November 2014, <<https://www.enisa.europa.eu/publications/algorithms-key-size-and-parameters-report-2014>>.
- [NIST] National Institute of Standards and Technology, "Recommendation for Key Management - Part 1: General", NIST Special Publication 800-57, Revision 4, DOI 10.6028/NIST.SP.800-57pt1r4, January 2016, <<http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r4.pdf>>.
- [PANOPTICCLICK] Electronic Frontier Foundation, "Panopticlick: Is your browser safe against tracking?", <<https://panopticlick.eff.org/>>.
- [RFC2246] Dierks, T. and C. Allen, "The TLS Protocol Version 1.0", RFC 2246, DOI 10.17487/RFC2246, January 1999, <<http://www.rfc-editor.org/info/rfc2246>>.

[RFC3526] Kivinen, T. and M. Kojo, "More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)", RFC 3526, DOI 10.17487/RFC3526, May 2003, <<http://www.rfc-editor.org/info/rfc3526>>.

[RFC4346] Dierks, T. and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.1", RFC 4346, DOI 10.17487/RFC4346, April 2006, <<http://www.rfc-editor.org/info/rfc4346>>.

[RFC4419] Friedl, M., Provos, N., and W. Simpson, "Diffie-Hellman Group Exchange for the Secure Shell (SSH) Transport Layer Protocol", RFC 4419, DOI 10.17487/RFC4419, March 2006, <<http://www.rfc-editor.org/info/rfc4419>>.

[RFC7027] Merkle, J. and M. Lochter, "Elliptic Curve Cryptography (ECC) Brainpool Curves for Transport Layer Security (TLS)", RFC 7027, DOI 10.17487/RFC7027, October 2013, <<http://www.rfc-editor.org/info/rfc7027>>.

[RFC7627] Bhargavan, K., Ed., Delignat-Lavaud, A., Pironti, A., Langley, A., and M. Ray, "Transport Layer Security (TLS) Session Hash and Extended Master Secret Extension", RFC 7627, DOI 10.17487/RFC7627, September 2015, <<http://www.rfc-editor.org/info/rfc7627>>.

[SECURE-RESUMPTION]
Delignat-Lavaud, A., Bhargavan, K., and A. Pironti, "Triple Handshakes Considered Harmful: Breaking and Fixing Authentication over TLS", 2014 IEEE Symposium on Security and Privacy, DOI 10.1109/SP.2014.14, March 2014, <<https://secure-resumption.com/>>.

[SSL3-ANALYSIS]
Schneier, B. and D. Wagner, "Analysis of the SSL 3.0 protocol", In Proceedings of the Second UNIX Workshop on Electronic Commerce, 1996, <<https://www.schneier.com/paper-ssl.pdf>>.

[STRONGSWAN-IKE]
Brunner, T. and A. Steffen, "IKEv2 Cipher Suites: Diffie Hellman Groups", October 2013, <<https://wiki.strongswan.org/projects/strongswan/wiki/IKEv2CipherSuites#Diffie-Hellman-Groups>>.

Appendix A. Supported Groups Registry (Formerly "EC Named Curve Registry")

Each description below indicates the group itself, its derivation, its expected strength (estimated roughly from guidelines in [ECRYPTII]), and whether it is recommended for use in TLS key exchange at the given security level. It is not recommended to add further finite field groups to the "Supported Groups Registry"; any attempt to do so should consider Section 9.1.

The primes in these finite field groups are all safe primes; that is, a prime p is a safe prime when $q = (p-1)/2$ is also prime. Where e is the base of the natural logarithm and square brackets denote the floor operation, the groups that initially populate this registry are derived for a given bit length b by finding the lowest positive integer X that creates a safe prime p where:

$$p = 2^b - 2^{\lfloor b-64 \rfloor} + \{ \lfloor 2^{\lfloor b-130 \rfloor} e \rfloor + X \} * 2^{64} - 1$$

New additions of FFDHE groups to this registry may use this same derivation (e.g., with different bit lengths) or may choose their parameters in a different way, but they must be clear about how the parameters were derived.

New additions of FFDHE groups MUST use a safe prime as the modulus to enable the inexpensive peer verification described in Section 5.1.

A.1. ffdhe2048

The 2048-bit group has registry value 256 and is calculated from the following formula:

The modulus is:

$$p = 2^{2048} - 2^{1984} + \{ \lfloor 2^{1918} * e \rfloor + 560316 \} * 2^{64} - 1$$

The hexadecimal representation of *p* is:

```
FFFFFFFF FFFFFFFF ADF85458 A2BB4A9A AFDC5620 273D3CF1
D8B9C583 CE2D3695 A9E13641 146433FB CC939DCE 249B3EF9
7D2FE363 630C75D8 F681B202 AEC4617A D3DF1ED5 D5FD6561
2433F51F 5F066ED0 85636555 3DED1AF3 B557135E 7F57C935
984F0C70 E0E68B77 E2A689DA F3EFE872 1DF158A1 36ADE735
30ACCA4F 483A797A BC0AB182 B324FB61 D108A94B B2C8E3FB
B96ADAB7 60D7F468 1D4F42A3 DE394DF4 AE56EDE7 6372BB19
0B07A7C8 EE0A6D70 9E02FCE1 CDF7E2EC C03404CD 28342F61
9172FE9C E98583FF 8E4F1232 EEF28183 C3FE3B1B 4C6FAD73
3BB5FCBC 2EC22005 C58EF183 7D1683B2 C6F34A26 C1B2EFFA
886B4238 61285C97 FFFFFFFF FFFFFFFF
```

The generator is: *g* = 2

The group size is: *q* = (*p*-1)/2

The hexadecimal representation of *q* is:

```
7FFFFFFFF FFFFFFFF D6FC2A2C 515DA54D 57EE2B10 139E9E78
EC5CE2C1 E7169B4A D4F09B20 8A3219FD E649CEE7 124D9F7C
BE97F1B1 B1863AEC 7B40D901 576230BD 69EF8F6A EAFEB2B0
9219FA8F AF833768 42B1B2AA 9EF68D79 DAAB89AF 3FABE49A
CC278638 707345BB F15344ED 79F7F439 0EF8AC50 9B56F39A
98566527 A41D3CBD 5E0558C1 59927DB0 E88454A5 D96471FD
DCB56D5B B06BFA34 0EA7A151 EF1CA6FA 572B76F3 B1B95D8C
8583D3E4 770536B8 4F017E70 E6FBF176 601A0266 941A17B0
C8B97F4E 74C2C1FF C7278919 777940C1 E1FF1D8D A637D6B9
9DDAFE5E 17611002 E2C778C1 BE8B41D9 6379A513 60D977FD
4435A11C 30942E4B FFFFFFFF FFFFFFFF
```

The estimated symmetric-equivalent strength of this group is 103 bits.

Peers using `ffdhe2048` that want to optimize their key exchange with a short exponent (Section 5.2) should choose a secret key of at least 225 bits.

A.2. ffdhe3072

The 3072-bit prime has registry value 257 and is calculated from the following formula:

The modulus is:

$$p = 2^{3072} - 2^{3008} + \{[2^{2942} * e] + 2625351\} * 2^{64} - 1$$

The hexadecimal representation of *p* is:

```
FFFFFFFF FFFFFFFF ADF85458 A2BB4A9A AFDC5620 273D3CF1
D8B9C583 CE2D3695 A9E13641 146433FB CC939DCE 249B3EF9
7D2FE363 630C75D8 F681B202 AEC4617A D3DF1ED5 D5FD6561
2433F51F 5F066ED0 85636555 3DED1AF3 B557135E 7F57C935
984F0C70 E0E68B77 E2A689DA F3EFE872 1DF158A1 36ADE735
30ACCA4F 483A797A BC0AB182 B324FB61 D108A94B B2C8E3FB
B96ADAB7 60D7F468 1D4F42A3 DE394DF4 AE56EDE7 6372BB19
0B07A7C8 EE0A6D70 9E02FCE1 CDF7E2EC C03404CD 28342F61
9172FE9C E98583FF 8E4F1232 EEF28183 C3FE3B1B 4C6FAD73
3BB5FCBC 2EC22005 C58EF183 7D1683B2 C6F34A26 C1B2EFFA
886B4238 611FCFDC DE355B3B 6519035B BC34F4DE F99C0238
61B46FC9 D6E6C907 7AD91D26 91F7F7EE 598CB0FA C186D91C
AEFE1309 85139270 B4130C93 BC437944 F4FD4452 E2D74DD3
64F2E21E 71F54BFF 5CAE82AB 9C9DF69E E86D2BC5 22363A0D
ABC52197 9B0DEADA 1DBF9A42 D5C4484E 0ABCD06B FA53DDEF
3C1B20EE 3FD59D7C 25E41D2B 66C62E37 FFFFFFFF FFFFFFFF
```

The generator is: *g* = 2

The group size is: *q* = (*p*-1)/2

The hexadecimal representation of *q* is:

```
7FFFFFFFF FFFFFFFF D6FC2A2C 515DA54D 57EE2B10 139E9E78
EC5CE2C1 E7169B4A D4F09B20 8A3219FD E649CEE7 124D9F7C
BE97F1B1 B1863AEC 7B40D901 576230BD 69EF8F6A EAFEB2B0
9219FA8F AF833768 42B1B2AA 9EF68D79 DAAB89AF 3FABE49A
CC278638 707345BB F15344ED 79F7F439 0EF8AC50 9B56F39A
98566527 A41D3CBD 5E0558C1 59927DB0 E88454A5 D96471FD
DCB56D5B B06BFA34 0EA7A151 EF1CA6FA 572B76F3 B1B95D8C
8583D3E4 770536B8 4F017E70 E6FBF176 601A0266 941A17B0
C8B97F4E 74C2C1FF C7278919 777940C1 E1FF1D8D A637D6B9
9DDAFE5E 17611002 E2C778C1 BE8B41D9 6379A513 60D977FD
4435A11C 308FE7EE 6F1AAD9D B28C81AD DE1A7A6F 7CCE011C
30DA37E4 EB736483 BD6C8E93 48FBFBF7 2CC6587D 60C36C8E
577F0984 C289C938 5A098649 DE21BCA2 7A7EA229 716BA6E9
B279710F 38FAA5FF AE574155 CE4EFB4F 743695E2 911B1D06
D5E290CB CD86F56D 0EDFCD21 6AE22427 055E6835 FD29EEF7
9E0D9077 1FEACEBE 12F20E95 B363171B FFFFFFFF FFFFFFFF
```

The estimated symmetric-equivalent strength of this group is 125 bits.

Peers using ffdhe3072 that want to optimize their key exchange with a short exponent (Section 5.2) should choose a secret key of at least 275 bits.

A.3. ffdhe4096

The 4096-bit group has registry value 258 and is calculated from the following formula:

The modulus is:

$$p = 2^{4096} - 2^{4032} + \{[2^{3966} * e] + 5736041\} * 2^{64} - 1$$

The hexadecimal representation of p is:

FFFFFFFF FFFFFFFF ADF85458 A2BB4A9A AFDC5620 273D3CF1
D8B9C583 CE2D3695 A9E13641 146433FB CC939DCE 249B3EF9
7D2FE363 630C75D8 F681B202 AEC4617A D3DF1ED5 D5FD6561
2433F51F 5F066ED0 85636555 3DED1AF3 B557135E 7F57C935
984F0C70 E0E68B77 E2A689DA F3EFE872 1DF158A1 36ADE735
30ACCA4F 483A797A BC0AB182 B324FB61 D108A94B B2C8E3FB
B96ADAB7 60D7F468 1D4F42A3 DE394DF4 AE56EDE7 6372BB19
0B07A7C8 EE0A6D70 9E02FCE1 CDF7E2EC C03404CD 28342F61
9172FE9C E98583FF 8E4F1232 EEF28183 C3FE3B1B 4C6FAD73
3BB5FCBC 2EC22005 C58EF183 7D1683B2 C6F34A26 C1B2EFFA
886B4238 611FCFDC DE355B3B 6519035B BC34F4DE F99C0238
61B46FC9 D6E6C907 7AD91D26 91F7F7EE 598CB0FA C186D91C
AEFE1309 85139270 B4130C93 BC437944 F4FD4452 E2D74DD3
64F2E21E 71F54BFF 5CAE82AB 9C9DF69E E86D2BC5 22363A0D
ABC52197 9B0DEADA 1DBF9A42 D5C4484E 0ABCD06B FA53DDEF
3C1B20EE 3FD59D7C 25E41D2B 669E1EF1 6E6F52C3 164DF4FB
7930E9E4 E58857B6 AC7D5F42 D69F6D18 7763CF1D 55034004
87F55BA5 7E31CC7A 7135C886 EFB4318A ED6A1E01 2D9E6832
A907600A 918130C4 6DC778F9 71AD0038 092999A3 33CB8B7A
1A1DB93D 7140003C 2A4ECEA9 F98D0ACC 0A8291CD CEC97DCF
8EC9B55A 7F88A46B 4DB5A851 F44182E1 C68A007E 5E655F6A
FFFFFFFF FFFFFFFF

The generator is: g = 2

The group size is: q = (p-1)/2

The hexadecimal representation of *q* is:

```
7FFFFFFF FFFFFFFF D6FC2A2C 515DA54D 57EE2B10 139E9E78
EC5CE2C1 E7169B4A D4F09B20 8A3219FD E649CEE7 124D9F7C
BE97F1B1 B1863AEC 7B40D901 576230BD 69EF8F6A EAFEB2B0
9219FA8F AF833768 42B1B2AA 9EF68D79 DAAB89AF 3FABE49A
CC278638 707345BB F15344ED 79F7F439 0EF8AC50 9B56F39A
98566527 A41D3CBD 5E0558C1 59927DB0 E88454A5 D96471FD
DCB56D5B B06BFA34 0EA7A151 EF1CA6FA 572B76F3 B1B95D8C
8583D3E4 770536B8 4F017E70 E6FBF176 601A0266 941A17B0
C8B97F4E 74C2C1FF C7278919 777940C1 E1FF1D8D A637D6B9
9DDAFE5E 17611002 E2C778C1 BE8B41D9 6379A513 60D977FD
4435A11C 308FE7EE 6F1AAD9D B28C81AD DE1A7A6F 7CCE011C
30DA37E4 EB736483 BD6C8E93 48FBFBF7 2CC6587D 60C36C8E
577F0984 C289C938 5A098649 DE21BCA2 7A7EA229 716BA6E9
B279710F 38FAA5FF AE574155 CE4EFB4F 743695E2 911B1D06
D5E290CB CD86F56D 0EDFCD21 6AE22427 055E6835 FD29EEF7
9E0D9077 1FEACEBE 12F20E95 B34F0F78 B737A961 8B26FA7D
BC9874F2 72C42BDB 563EAF1 6B4FB68C 3BB1E78E AA81A002
43FAADD2 BF18E63D 389AE443 77DA18C5 76B50F00 96CF3419
5483B005 48C09862 36E3BC7C B8D6801C 0494CCD1 99E5C5BD
0D0EDC9E B8A0001E 15276754 FCC68566 054148E6 E764BEE7
C764DAAD 3FC45235 A6DAD428 FA20C170 E345003F 2F32AFB5
7FFFFFFF FFFFFFFF
```

The estimated symmetric-equivalent strength of this group is 150 bits.

Peers using ffdhe4096 that want to optimize their key exchange with a short exponent (Section 5.2) should choose a secret key of at least 325 bits.

A.4. ffdhe6144

The 6144-bit group has registry value 259 and is calculated from the following formula:

The modulus is:

$$p = 2^{6144} - 2^{6080} + \{[2^{6014} * e] + 15705020\} * 2^{64} - 1$$

The hexadecimal representation of p is:

FFFFFFFF	FFFFFFFF	ADF85458	A2BB4A9A	AFDC5620	273D3CF1
D8B9C583	CE2D3695	A9E13641	146433FB	CC939DCE	249B3EF9
7D2FE363	630C75D8	F681B202	AEC4617A	D3DF1ED5	D5FD6561
2433F51F	5F066ED0	85636555	3DED1AF3	B557135E	7F57C935
984F0C70	E0E68B77	E2A689DA	F3EFE872	1DF158A1	36ADE735
30ACCA4F	483A797A	BC0AB182	B324FB61	D108A94B	B2C8E3FB
B96ADAB7	60D7F468	1D4F42A3	DE394DF4	AE56EDE7	6372BB19
0B07A7C8	EE0A6D70	9E02FCE1	CDF7E2EC	C03404CD	28342F61
9172FE9C	E98583FF	8E4F1232	EEF28183	C3FE3B1B	4C6FAD73
3BB5FCBC	2EC22005	C58EF183	7D1683B2	C6F34A26	C1B2EFFA
886B4238	611FCFDC	DE355B3B	6519035B	BC34F4DE	F99C0238
61B46FC9	D6E6C907	7AD91D26	91F7F7EE	598CB0FA	C186D91C
AEFE1309	85139270	B4130C93	BC437944	F4FD4452	E2D74DD3
64F2E21E	71F54BFF	5CAE82AB	9C9DF69E	E86D2BC5	22363A0D
ABC52197	9B0DEADA	1DBF9A42	D5C4484E	0ABCD06B	FA53DDEF
3C1B20EE	3FD59D7C	25E41D2B	669E1EF1	6E6F52C3	164DF4FB
7930E9E4	E58857B6	AC7D5F42	D69F6D18	7763CF1D	55034004
87F55BA5	7E31CC7A	7135C886	EFB4318A	ED6A1E01	2D9E6832
A907600A	918130C4	6DC778F9	71AD0038	092999A3	33CB8B7A
1A1DB93D	7140003C	2A4ECEA9	F98D0ACC	0A8291CD	CEC97DCF
8EC9B55A	7F88A46B	4DB5A851	F44182E1	C68A007E	5E0DD902
0BFD64B6	45036C7A	4E677D2C	38532A3A	23BA4442	CAF53EA6
3BB45432	9B7624C8	917BDD64	B1C0FD4C	B38E8C33	4C701C3A
CDAD0657	FCCFEC71	9B1F5C3E	4E46041F	388147FB	4CFDB477
A52471F7	A9A96910	B855322E	DB6340D8	A00EF092	350511E3
0ABEC1FF	F9E3A26E	7FB29F8C	183023C3	587E38DA	0077D9B4
763E4E4B	94B2BBC1	94C6651E	77CAF992	EEAAC023	2A281BF6
B3A739C1	22611682	0AE8DB58	47A67CBE	F9C9091B	462D538C
D72B0374	6AE77F5E	62292C31	1562A846	505DC82D	B854338A
E49F5235	C95B9117	8CCF2DD5	CACEF403	EC9D1810	C6272B04
5B3B71F9	DC6B80D6	3FDD4A8E	9ADB1E69	62A69526	D43161C1
A41D570D	7938DAD4	A40E329C	D0E40E65	FFFFFFFF	FFFFFFFF

The generator is: $g = 2$

The group size is: $q = (p-1)/2$

The hexadecimal representation of *q* is:

7FFFFFFF FFFFFFFF D6FC2A2C 515DA54D 57EE2B10 139E9E78
EC5CE2C1 E7169B4A D4F09B20 8A3219FD E649CEE7 124D9F7C
BE97F1B1 B1863AEC 7B40D901 576230BD 69EF8F6A EAFEB2B0
9219FA8F AF833768 42B1B2AA 9EF68D79 DAAB89AF 3FABE49A
CC278638 707345BB F15344ED 79F7F439 0EF8AC50 9B56F39A
98566527 A41D3CBD 5E0558C1 59927DB0 E88454A5 D96471FD
DCB56D5B B06BFA34 0EA7A151 EF1CA6FA 572B76F3 B1B95D8C
8583D3E4 770536B8 4F017E70 E6FBF176 601A0266 941A17B0
C8B97F4E 74C2C1FF C7278919 777940C1 E1FF1D8D A637D6B9
9DDAFE5E 17611002 E2C778C1 BE8B41D9 6379A513 60D977FD
4435A11C 308FE7EE 6F1AAD9D B28C81AD DE1A7A6F 7CCE011C
30DA37E4 EB736483 BD6C8E93 48FBFBF7 2CC6587D 60C36C8E
577F0984 C289C938 5A098649 DE21BCA2 7A7EA229 716BA6E9
B279710F 38FAA5FF AE574155 CE4EFB4F 743695E2 911B1D06
D5E290CB CD86F56D 0EDFCD21 6AE22427 055E6835 FD29EEF7
9E0D9077 1FEACEBE 12F20E95 B34F0F78 B737A961 8B26FA7D
BC9874F2 72C42BDB 563EAF1 6B4FB68C 3BB1E78E AA81A002
43FAADD2 BF18E63D 389AE443 77DA18C5 76B50F00 96CF3419
5483B005 48C09862 36E3BC7C B8D6801C 0494CCD1 99E5C5BD
0D0EDC9E B8A0001E 15276754 FCC68566 054148E6 E764BEE7
C764DAAD 3FC45235 A6DAD428 FA20C170 E345003F 2F06EC81
05FEB25B 2281B63D 2733BE96 1C29951D 11DD2221 657A9F53
1DDA2A19 4DBB1264 48BDEEB2 58E07EA6 59C74619 A6380E1D
66D6832B FE67F638 CD8FAE1F 2723020F 9C40A3FD A67EDA3B
D29238FB D4D4B488 5C2A9917 6DB1A06C 50077849 1A8288F1
855F60FF FCF1D137 3FD94FC6 0C1811E1 AC3F1C6D 003BECDA
3B1F2725 CA595DE0 CA63328F 3BE57CC9 77556011 95140DFB
59D39CE0 91308B41 05746DAC 23D33E5F 7CE4848D A316A9C6
6B9581BA 3573BFAF 31149618 8AB15423 282EE416 DC2A19C5
724FA91A E4ADC88B C66796EA E5677A01 F64E8C08 63139582
2D9DB8FC EE35C06B 1FEEA547 4D6D8F34 B1534A93 6A18B0E0
D20EAB86 BC9C6D6A 5207194E 68720732 FFFFFFFF FFFFFFFF

The estimated symmetric-equivalent strength of this group is 175 bits.

Peers using ffdhe6144 that want to optimize their key exchange with a short exponent (Section 5.2) should choose a secret key of at least 375 bits.

A.5. ffdhe8192

The 8192-bit group has registry value 260 and is calculated from the following formula:

The modulus is:

$$p = 2^{8192} - 2^{8128} + \{[2^{8062} * e] + 10965728\} * 2^{64} - 1$$

The hexadecimal representation of p is:

FFFFFFFF FFFFFFFF ADF85458 A2BB4A9A AFDC5620 273D3CF1
D8B9C583 CE2D3695 A9E13641 146433FB CC939DCE 249B3EF9
7D2FE363 630C75D8 F681B202 AEC4617A D3DF1ED5 D5FD6561
2433F51F 5F066ED0 85636555 3DED1AF3 B557135E 7F57C935
984F0C70 E0E68B77 E2A689DA F3EFE872 1DF158A1 36ADE735
30ACCA4F 483A797A BC0AB182 B324FB61 D108A94B B2C8E3FB
B96ADAB7 60D7F468 1D4F42A3 DE394DF4 AE56EDE7 6372BB19
0B07A7C8 EE0A6D70 9E02FCE1 CDF7E2EC C03404CD 28342F61
9172FE9C E98583FF 8E4F1232 EEF28183 C3FE3B1B 4C6FAD73
3BB5FCBC 2EC22005 C58EF183 7D1683B2 C6F34A26 C1B2EFFA
886B4238 611FCFDC DE355B3B 6519035B BC34F4DE F99C0238
61B46FC9 D6E6C907 7AD91D26 91F7F7EE 598CB0FA C186D91C
AEFE1309 85139270 B4130C93 BC437944 F4FD4452 E2D74DD3
64F2E21E 71F54BFF 5CAE82AB 9C9DF69E E86D2BC5 22363A0D
ABC52197 9B0DEADA 1DBF9A42 D5C4484E 0ABCD06B FA53DDEF
3C1B20EE 3FD59D7C 25E41D2B 669E1EF1 6E6F52C3 164DF4FB
7930E9E4 E58857B6 AC7D5F42 D69F6D18 7763CF1D 55034004
87F55BA5 7E31CC7A 7135C886 EFB4318A ED6A1E01 2D9E6832
A907600A 918130C4 6DC778F9 71AD0038 092999A3 33CB8B7A
1A1DB93D 7140003C 2A4ECEA9 F98D0ACC 0A8291CD CEC97DCF
8EC9B55A 7F88A46B 4DB5A851 F44182E1 C68A007E 5E0DD902
0BFD64B6 45036C7A 4E677D2C 38532A3A 23BA4442 CAF53EA6
3BB45432 9B7624C8 917BDD64 B1C0FD4C B38E8C33 4C701C3A
CDAD0657 FCCFEC71 9B1F5C3E 4E46041F 388147FB 4CFDB477
A52471F7 A9A96910 B855322E DB6340D8 A00EF092 350511E3
0ABEC1FF F9E3A26E 7FB29F8C 183023C3 587E38DA 0077D9B4
763E4E4B 94B2BBC1 94C6651E 77CAF992 EEAAC023 2A281BF6
B3A739C1 22611682 0AE8DB58 47A67CBE F9C9091B 462D538C
D72B0374 6AE77F5E 62292C31 1562A846 505DC82D B854338A
E49F5235 C95B9117 8CCF2DD5 CACEF403 EC9D1810 C6272B04
5B3B71F9 DC6B80D6 3FDD4A8E 9ADB1E69 62A69526 D43161C1
A41D570D 7938DAD4 A40E329C CFF46AAA 36AD004C F600C838
1E425A31 D951AE64 FDB23FCE C9509D43 687FEB69 EDD1CC5E
0B8CC3BD F64B10EF 86B63142 A3AB8829 555B2F74 7C932665
CB2C0F1C C01BD702 29388839 D2AF05E4 54504AC7 8B758282
2846C0BA 35C35F5C 59160CC0 46FD8251 541FC68C 9C86B022
BB709987 6A460E74 51A8A931 09703FEE 1C217E6C 3826E52C
51AA691E 0E423CFC 99E9E316 50C1217B 624816CD AD9A95F9
D5B80194 88D9C0A0 A1FE3075 A577E231 83F81D4A 3F2FA457
1EFC8CE0 BA8A4FE8 B6855DFE 72B0A66E DED2FBAB FBE58A30
FAFABE1C 5D71A87E 2F741EF8 C1FE86FE A6BBFDE5 30677F0D
97D11D49 F7A8443D 0822E506 A9F4614E 011E2A94 838FF88C
D68C8BB7 C5C6424C FFFFFFFF FFFFFFFF

The generator is: g = 2

The group size is: $q = (p-1)/2$

The hexadecimal representation of q is:

7FFFFFFF	FFFFFFFF	D6FC2A2C	515DA54D	57EE2B10	139E9E78
EC5CE2C1	E7169B4A	D4F09B20	8A3219FD	E649CEE7	124D9F7C
BE97F1B1	B1863AEC	7B40D901	576230BD	69EF8F6A	EAFEB2B0
9219FA8F	AF833768	42B1B2AA	9EF68D79	DAAB89AF	3FABE49A
CC278638	707345BB	F15344ED	79F7F439	0EF8AC50	9B56F39A
98566527	A41D3CBD	5E0558C1	59927DB0	E88454A5	D96471FD
DCB56D5B	B06BFA34	0EA7A151	EF1CA6FA	572B76F3	B1B95D8C
8583D3E4	770536B8	4F017E70	E6FBF176	601A0266	941A17B0
C8B97F4E	74C2C1FF	C7278919	777940C1	E1FF1D8D	A637D6B9
9DDAFE5E	17611002	E2C778C1	BE8B41D9	6379A513	60D977FD
4435A11C	308FE7EE	6F1AAD9D	B28C81AD	DE1A7A6F	7CCE011C
30DA37E4	EB736483	BD6C8E93	48FBFBF7	2CC6587D	60C36C8E
577F0984	C289C938	5A098649	DE21BCA2	7A7EA229	716BA6E9
B279710F	38FAA5FF	AE574155	CE4EFB4F	743695E2	911B1D06
D5E290CB	CD86F56D	0EDFCD21	6AE22427	055E6835	FD29EEF7
9E0D9077	1FEACEBE	12F20E95	B34F0F78	B737A961	8B26FA7D
BC9874F2	72C42BDB	563EAFa1	6B4FB68C	3BB1E78E	AA81A002
43FAADD2	BF18E63D	389AE443	77DA18C5	76B50F00	96CF3419
5483B005	48C09862	36E3BC7C	B8D6801C	0494CCD1	99E5C5BD
0D0EDC9E	B8A0001E	15276754	FCC68566	054148E6	E764BEE7
C764DAAD	3FC45235	A6DAD428	FA20C170	E345003F	2F06EC81
05FEB25B	2281B63D	2733BE96	1C29951D	11DD2221	657A9F53
1DDA2A19	4DBB1264	48BDEEB2	58E07EA6	59C74619	A6380E1D
66D6832B	FE67F638	CD8FAE1F	2723020F	9C40A3FD	A67EDA3B
D29238FB	D4D4B488	5C2A9917	6DB1A06C	50077849	1A8288F1
855F60FF	FCF1D137	3FD94FC6	0C1811E1	AC3F1C6D	003BECDA
3B1F2725	CA595DE0	CA63328F	3BE57CC9	77556011	95140DFB
59D39CE0	91308B41	05746DAC	23D33E5F	7CE4848D	A316A9C6
6B9581BA	3573BFAF	31149618	8AB15423	282EE416	DC2A19C5
724FA91A	E4ADC88B	C66796EA	E5677A01	F64E8C08	63139582
2D9DB8FC	EE35C06B	1FEEA547	4D6D8F34	B1534A93	6A18B0E0
D20EAB86	BC9C6D6A	5207194E	67FA3555	1B568026	7B00641C
0F212D18	ECA8D732	7ED91FE7	64A84EA1	B43FF5B4	F6E8E62F
05C661DE	FB258877	C35B18A1	51D5C414	AAAD97BA	3E499332
E596078E	600DEB81	149C441C	E95782F2	2A282563	C5BAC141
1423605D	1AE1AFAE	2C8B0660	237EC128	AA0FE346	4E435811
5DB84CC3	B523073A	28D45498	84B81FF7	0E10BF36	1C137296
28D5348F	07211E7E	4CF4F18B	286090BD	B1240B66	D6CD4AFC
EADC00CA	446CE050	50FF183A	D2BBF118	C1FC0EA5	1F97D22B
8F7E4670	5D4527F4	5B42AEFF	39585337	6F697DD5	FDF2C518
7D7D5F0E	2EB8D43F	17BA0F7C	60FF437F	535DFEF2	9833BF86
CBE88EA4	FBD4221E	84117283	54FA30A7	008F154A	41C7FC46
6B4645DB	E2E32126	7FFFFFFF	FFFFFFFF		

The estimated symmetric-equivalent strength of this group is 192 bits.

Peers using ffdhe8192 that want to optimize their key exchange with a short exponent (Section 5.2) should choose a secret key of at least 400 bits.

Acknowledgements

Thanks to Fedor Brunner, Dave Fergemann, Niels Ferguson, Sandy Harris, Tero Kivinen, Watson Ladd, Nikos Mavrogiannopolous, Niels Moeller, Bodo Moeller, Kenny Paterson, Eric Rescorla, Tom Ritter, Rene Struik, Martin Thomson, Sean Turner, and other members of the TLS Working Group for their comments and suggestions on this document. Any mistakes here are not theirs.

Author's Address

Daniel Kahn Gillmor
ACLU
125 Broad Street, 18th Floor
New York, NY 10004
United States of America

Email: dkg@fifthhorseman.net